

Predictive Modeling of Stock Prices Using SVR with Kernel Evaluation and GA Optimization

Buu Quan Luu

August 15, 2025

Keywords: Support Vector Regression, SVR, Genetic Algorithm, Stock Prediction, Machine Learning, Financial Forecasting, Kernel Methods

1 Introduction

In any country, the stock market represents a complex and vital component of the economic system, engaging millions of investors and institutions worldwide. Understanding market behavior is crucial for stakeholders, as financial markets exhibit increasing volatility driven by nonlinear dependencies, geopolitical factors, and rapidly changing investor sentiment [1]. Accurate stock price prediction has consequently become a critical research focus, offering investors and policymakers the ability to identify trends, optimize portfolios, and mitigate risks. However, the non-stationary and noisy nature of financial data makes this an exceptionally challenging task for traditional statistical models, creating significant opportunities for machine intelligence to enhance forecasting accuracy through advanced pattern recognition and real-time adaptive learning.

The motivation for this research stems from both the practical importance of financial forecasting and the technical challenges posed by market dynamics. Stock prices are influenced by a complex interplay of quantitative factors (e.g., historical trends, trading volumes) and qualitative signals (e.g., news sentiment, earnings reports), which conventional time-series models often fail to capture effectively [2]. Traditional forecasting methods often struggle to account for these complexities, leading to inconsistent or unreliable predictions.

This limitation has spurred growing interest in machine learning approaches, specifically in the domain of applying the supervised learning in Support Vector Machine (SVM) algorithm. In this case, the objective is to forecast the adjusted close price of Google stock using historical data from

2020 to 2024. Supervised learning is a machine learning approach where models are trained on labeled data—input features paired with known outputs—to learn patterns and make predictions [3]. It involves two key phases: training, where the model learns from historical data, and testing, where it makes predictions on unseen data [3]. Supervised learning is commonly used in tasks like classification (discrete labels) and regression (continuous values), making it ideal for problems such as predicting stock prices or categorizing objects. Specifically, it applies Support Vector Regression (SVR) using multiple kernel functions (Linear, Polynomial, RBF) to assess predictive performance and model evaluation as part of a comprehensive machine learning pipeline.

2 Methodology

2.1 Model Selection

Support Vector Regression (SVR) is a robust and versatile machine learning technique derived from the principles of Support Vector Machines (SVM) [4]. The algorithm strives to find a function that fits within this margin while maintaining minimal complexity, called hyperplane. The SVM algorithm detects the weight vector w and the bias term b that define the hyperplane in equation:

$$y = \omega \cdot \mathbf{x} + b \quad (1)$$

SVM seeks to find the optimal hyperplane that best separates the two classes. The optimal hyperplane is the one that maximizes the margin, which is the distance between the hyperplane and the closest data points of each class. This balance makes SVR especially powerful for stock price prediction, where the data tends to be noisy, volatile, and influenced by numerous unpredictable external factors. SVR is its ability to model both linear and non-

linear relationships using kernel functions, which measure similarity between data points in transformed feature spaces.

2.1.1 Linear Kernel

The linear kernel assumes that the relationship between input features and the output is straightforward and linearly separable. It computes a simple dot product between feature vectors and is ideal when the data exhibits a strong linear trend. While it's computationally efficient and interpretable, it may underperform on highly volatile or non-linear datasets like stock prices.

2.1.2 Polynomial Kernel

The polynomial kernel is well-suited for problems where the relationship between variables is more complex but still follows a curvilinear pattern. It maps input features into higher dimensions using polynomial transformations, allowing the model to fit more flexible regression curves. This kernel is useful when moderate non-linear interactions exist in the data.

2.1.3 Radial Basis Function (RBF) Kernel

The RBF kernel is highly effective for capturing intricate, non-linear patterns. It projects the data into an infinite-dimensional space, making it capable of fitting even the most irregular data structures. This is particularly beneficial in financial forecasting, where price movements are influenced by countless subtle and interconnected variables. The RBF kernel's flexibility makes it the most suitable choice for modelling real-world stock data with high variability.

2.2 Dataset and Preprocessing steps

In the dataset, there are 7 columns in total, named 'Date', 'Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume', which are the recording date of the stock, the adjusted close price, the close price, the highest price of the day, the lowest price of the day, the open price, and the trading volume in the day respectively. This project addresses the problem of predicting the adjusted close price of Google (GOOGL) stock using historical stock market data from 2020 to 2025, which is collected from the Kaggle. The adjusted close price represents for the

stock's value after the deduction from corporate actions, for examples, stock splits, dividends payout, which is the last price during the regular trading day. (FIGURE 1)

The preprocessing began with numerous approaches by loading the dataset and conducting exploratory analysis to understand its structure, check data types, and compute summary statistics. Missing values and duplicate records were identified and handled to ensure data quality. The 'Date' column was converted to a proper datetime format, and additional temporal features were extracted—namely Year, Month, Day of Month, and Day of Week—to enable time-series pattern recognition. Visualizations such as line plots were generated to examine trends in the adjusted closing price over time. Boxplots and other visual tools were used to highlight variations in price across different years, months, and days of the week. Furthermore, correlation analysis was conducted to assess relationships between all the features in the dataset. (FIGURE 2)

These preprocessing steps in the figure are to focus on the visualization of the patterns of the adjusted close price during the period from 2020 to 2024, which is recorded an increase in the trend of the Google's stock prices over time. (FIGURE 3)

Then to see the trend apparently, The Date column was converted to a datetime format, and new features such as Year, Month, and DayOfWeek were extracted to analyse trends. In the next figure, the box plots illustrate the mean of adjusted price by year, month and day of week. (FIGURE 4)

Next, performing the correlation matrix in the heat map highlights the importance of understanding features relationships to guide the model selection for better predictive performance. From the figure, the 'adjust close' column is impacted by almost other features, with the highly positive correlated. (FIGURE 5)

3 Implementation

The development and implementation of the stock price prediction model were carried out using the Python programming language, which is the primary language for backend logic, data processing, due to its flexibility and the extensive range of data science libraries it offers. The project was developed within a Jupyter Notebook environment integrated into Visual Studio Code (VS Code), which

enabled an efficient workflow for interactive coding, visualization, and debugging. A suite of specialized libraries supported various stages of the process. The pandas library was used for data ingestion, cleaning, and manipulation, while numpy facilitated numerical operations and array-based transformations. To construct and evaluate the Support Vector Regression (SVR) models, the scikit-learn library was employed, offering a wide selection of machine learning tools. For data visualization, both matplotlib and seaborn were used to generate static plots that helped identify patterns and trends within the dataset. Additionally, the plotly library was utilized to create interactive plots, enabling a more dynamic and detailed temporal analysis of stock price behaviour. This combination of tools provided a comprehensive and efficient environment for building and assessing the machine learning models.

To conducting the model, the implementation of feature engineering and data splitting for the Support Vector Regression (SVR) model began with extracting temporal features from the Date column. The column was converted to Python’s date-time format, resulting in three new features—Year, Month, and DayOfMonth—that capture temporal patterns such as seasonality and trends over time. To enhance the model’s learning from historical data, lagged features were introduced, including Adj Close Lag1 and Adj Close Lag2 for the adjusted closing prices from the previous one and two days, respectively, and Volume Lag1 for the trading volume from the day before. These lagged variables are vital for financial forecasting, as they significantly influence future trends. Following the introduction of lagged features, initial rows containing NaN values were removed to ensure data integrity. The dataset was then chronologically split into training and testing subsets: data from January 2020 to December 2023 was used for training, while data from January 2024 onward was reserved for testing, thus reflecting a realistic forecasting scenario and preventing data leakage. Finally, a set of ten features, incorporating both raw market indicators and engineered variables, was selected as the input (x), while the adjusted closing price (Adj Close) served as the target variable (y), facilitating the supervised learning process. StandardScaler was effectively utilized to normalize numerical inputs, including High, Low, Open, Volume, and their lagged values. This normalization process contributed to more efficient convergence of the SVR model, resulting in enhanced reliability of

the predictions generated

3.1 SVR

In this study, SVR stands as the foundational model. Trained with default hyperparameters and without any optimization, the SVR model provides a valuable benchmark for assessing the performance of other models. This approach enhances the clarity of our evaluations and underscores the efficiencies of the alternatives [5].

Table 1: SVR Hyperparameter Configuration

Parameter	Kernel Type		
	RBF	Linear	Polynomial
C	1	1	1
Epsilon	0.1	0.1	0.1
gamma	Scale	Scale	Scale

The linear kernel assumes a direct linear relationship between input and target variables, aligning well with the dataset’s structure, which enhances its effectiveness in such scenarios. In contrast, the RBF and polynomial kernels are designed to capture non-linear relationships but tend to perform poorly with default parameters. These kernels are particularly sensitive to the choice of hyperparameters, especially C, gamma, and epsilon. Inappropriate defaults can lead either to underfitting or overfitting, particularly in datasets where the underlying patterns are predominantly linear. This highlights the importance of hyperparameter optimization, especially when utilizing non-linear kernels.

3.2 Genetic Algorithm - SVR

The Genetic Algorithm (GA) was utilized as a metaheuristic optimization technique to fine-tune the hyperparameters of a Support Vector Regression (SVR) model. Specifically, three crucial hyperparameters—C (the regularization parameter), gamma (the kernel coefficient), and epsilon (the margin of tolerance)—were optimized within predefined bounds using GA. The objective function aimed to minimize the mean squared error (MSE) on the test set and optimise the R2, thereby steering the evolutionary search process towards the most effective parameter combinations. Once the optimal parameters were identified, they were applied to train three distinct kernel in SVR models. The models were trained on scaled features using

StandardScaler and were evaluated based on their ability to predict the 'Adj Close' stock prices.

Table 2: Genetic Algorithm Hyperparameter Search Space

Parameter	Search Range
C (Regularization)	0.1 to 1000
γ (Kernel coefficient)	0.0001 to 1
ϵ (Margin tolerance)	0.001 to 1

Table 3: Genetic Algorithm Configuration Parameters

Parameter	Value
Maximum iterations	50
Population size	20
Mutation probability	0.1
Elitism ratio	0.01
Crossover probability	0.5
Parents portion	0.3
Crossover type	Uniform
Max iterations without improvement	10

4 Results and Discussion

4.1 Experimental Results

The performance of Support Vector Regression (SVR) models with different kernels—RBF, linear, and polynomial—was evaluated on stock price prediction. The models were first assessed using default parameters and later with hyperparameters optimized using a Genetic Algorithm (GA). The evaluation metrics employed were Mean Squared Error (MSE) and R-squared (R^2), measured on the testing set representing the year 2024.

A time-based data split was used, allocating 80 percent of the data (2020–2023) for training and 20 percent (2024) for testing to ensure temporal consistency in forecasting performance. (FIGURE 6, FIGURE 7)

Table 4: Performance Comparison Before and After Genetic Algorithm Optimization

Kernel Type	MSE (Before GA)	R^2 (Before GA)	MSE (After GA)	R^2 (After GA)
RBF	3228.89	-12.5592	18.02	0.9243
Linear	1.35	0.9943	0.999	0.9958
Polynomial	14151.42	-58.4267	1148.52	-3.8230

The optimal hyperparameters found by the Genetic Algorithm were: $C = 860.25$, $\gamma =$

0.0030 , $\epsilon = 0.0285$. These optimized values significantly improved the RBF kernel, reducing the MSE from over 3000 to just 18, and improving R^2 from a highly negative value to 0.924, indicating strong explanatory power post-optimization. The linear kernel, which was already performing well due to the linear nature of the dataset, showed slight improvements. However, despite optimization, the polynomial kernel continued to underperform, suggesting that the dataset lacks the non-linear patterns necessary for polynomial kernel effectiveness. (FIGURE 6, FIGURE 8)

4.2 Effectiveness Analysis

Support Vector Regression (SVR) was chosen for its ability to manage both linear and non-linear relationships, which is vital for modeling complex financial data like stock prices. The standard SVR model with a linear kernel showed strong predictive performance due to feature correlation. However, the RBF and polynomial kernels struggled with default hyperparameters, revealing SVR’s sensitivity to parameter settings. To improve this, a Genetic Algorithm (GA) was employed to optimize key hyperparameters (C , γ , ϵ). This approach significantly enhanced the RBF kernel’s performance, boosting its predictive power from ($R^2 = -12.56$) to ($R^2 = 0.92$). Despite these gains, the polynomial kernel remained underwhelming, indicating that SVR’s effectiveness depends on data characteristics and kernel compatibility. While GA optimization increased computational costs, it proved to be a valuable solution for overcoming the challenges of manual tuning in SVR for time series stock prediction.

5 Conclusion

This project demonstrated the effectiveness of Support Vector Regression (SVR) models in forecasting stock prices using time series data. Among the three evaluated kernels—linear, RBF, and polynomial—the linear kernel initially outperformed the others due to its strong feature correlation. However, by integrating a Genetic Algorithm (GA) to optimize the hyperparameters of the SVR, the performance of the RBF kernel significantly improved, achieving high accuracy and surpassing its default configuration. This underscores the importance of automated hyperparameter tuning in enhancing model performance.

Ethical considerations arise from the potential misuse of predictive financial models for market manipulation or insider trading. While the model relies on historical data for forecasting, it does not account for unpredictable events (such as political turmoil or economic crises), which may limit its real-world reliability. Furthermore, the computational demands of GA and SVR may not be suitable for low-resource environments or real-time applications.

Future work could explore the incorporation of more domain-specific financial indicators, such as technical analysis metrics or macroeconomic variables, to enrich the feature set. Additionally, integrating advanced models such as Long Short-Term Memory (LSTM) networks or hybrid ensembles could further enhance predictive capabilities. Applying explainability techniques would also provide transparency in decision-making, thereby supporting ethical and interpretable AI in finance.

Appendix

Visualizations

	Date	Adj Close	Close	High	Low	Open	Volume
0	2020-01-02 00:00:00+00:00	68.186821	68.433998	68.433998	67.324501	67.420502	27278000
1	2020-01-03 00:00:00+00:00	67.830101	68.075996	68.687500	67.365997	67.400002	23408000
2	2020-01-06 00:00:00+00:00	69.638054	69.890503	69.916000	67.550003	67.581497	46768000
3	2020-01-07 00:00:00+00:00	69.503548	69.755501	70.175003	69.578003	70.023003	34330000
4	2020-01-08 00:00:00+00:00	69.998253	70.251999	70.592499	69.631500	69.740997	35314000
...	...	...	...	...	...	...	...
1253	2024-12-24 00:00:00+00:00	196.110001	196.110001	196.110001	193.779999	194.839996	10403300
1254	2024-12-26 00:00:00+00:00	195.600006	195.600006	196.750000	194.380005	195.149994	12046600
1255	2024-12-27 00:00:00+00:00	192.759995	192.759995	195.320007	190.649994	194.949997	18891400
1256	2024-12-30 00:00:00+00:00	191.240005	191.240005	192.550003	189.119995	189.800003	14264700
1257	2024-12-31 00:00:00+00:00	189.300003	189.300003	191.960007	188.509995	191.080002	17466900

1258 rows x 7 columns

Figure 1: The structure of the Google stock price dataset

```
# Check for missing values
print (dataset.isnull().sum())

# Check for duplicates
print (dataset.duplicated().sum())

# Convert the 'Date' column to datetime format
dataset['Date'] = pd.to_datetime(dataset['Date']).dt.date
dataset['Date'] = pd.to_datetime(dataset['Date'], format='%Y-%m-%d')
```

Figure 2: Dataset preprocessing workflow



Figure 3: Overall trend of the Adjusted close price from 2020 to 2024

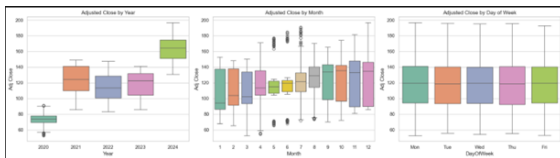


Figure 4: Trend adjusted close price by Year, Month, and Day of Week

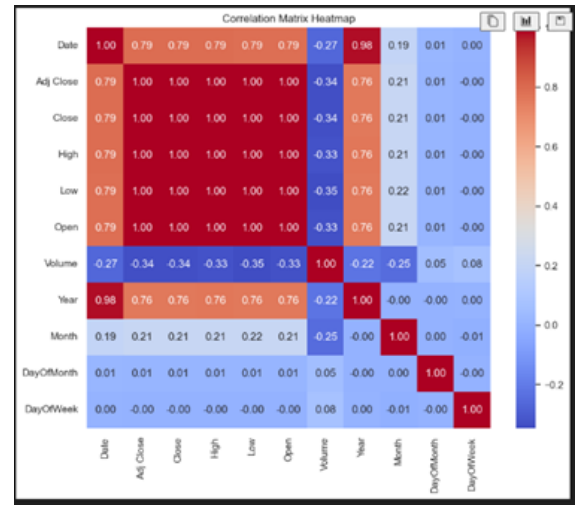


Figure 5: Correlation Matrix Heatmap

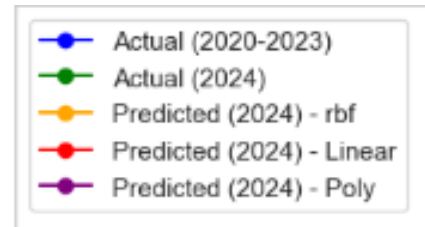


Figure 6: The definition of the line



Figure 7: SVR model for each kernel with default parameters



Figure 8: SVR model for each kernel with Genetic Algorithm

References

- [1] P. Meesad and R. I. Rasel, “Predicting stock market price using support vector regression,” in *2013 International Conference on Informatics, Electronics and Vision (ICIEV)*, 2013, pp. 1–6.
- [2] K. jae Kim, “Financial time series forecasting using support vector machines,” *Neurocomputing*, vol. 55, no. 1, pp. 307–319, 2003, support Vector Machines. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231203003722>
- [3] V. Nasteski, “An overview of the supervised machine learning methods,” *Horizons. b*, vol. 4, no. 51-62, p. 56, 2017.
- [4] D. A. Pisner and D. M. Schnyer, “Support vector machine,” in *Machine learning*. Elsevier, 2020, pp. 101–121.
- [5] M. Beniwal, A. Singh, and N. Kumar, “Forecasting long-term stock prices of global indices: A forward-validating genetic algorithm optimization approach for support vector regression,” *Applied Soft Computing*, vol. 145, p. 110566, 2023.